



V0088

Advanced Information Security

高级信息安全

Instructor: **Prof. Xiaohong Yu**

Graduate School of Information and Communication Technology,
Ajou University, Korea

密码学

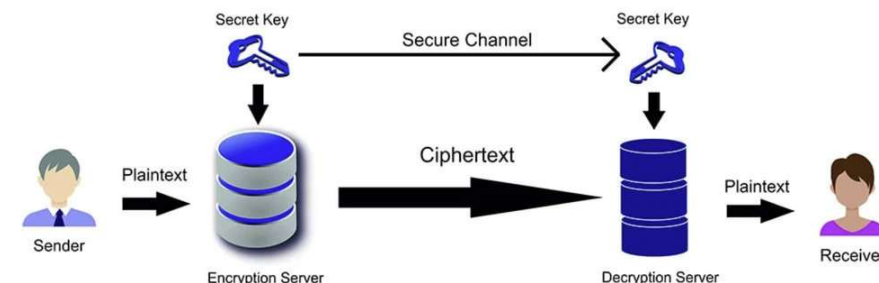
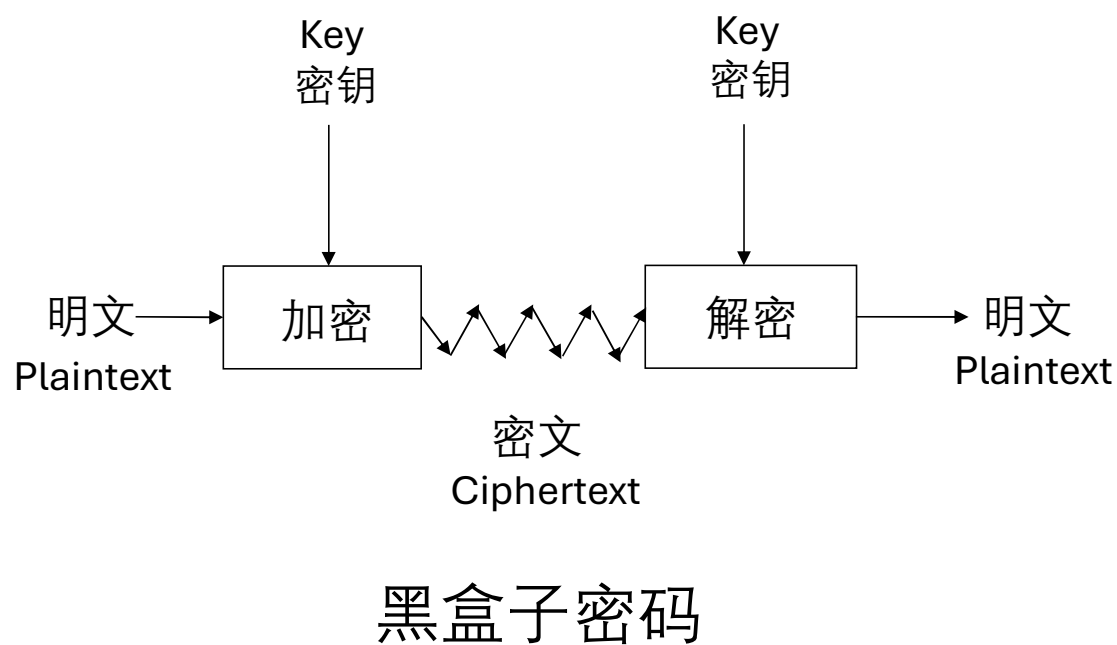
- 密码学基础
- 对称密钥密码
- 公钥密码
- Hash 函数及其他密码
- 高级密码分析

密码学

- 密码学基础（简介，含义，古典密码，现代密码，以及分类）
- 对称密钥密码（流密码，分组密码和完整性）
- 公钥密码
- Hash 函数及其他密码
- 高级密码分析

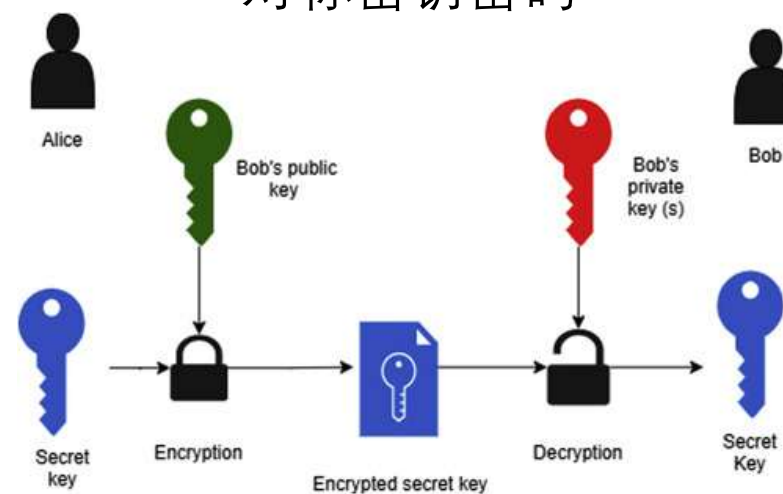
第一部分：密码学基础

密码学



Symmetric Cryptography

对称密钥密码



非对称密钥密码

柯克霍夫原理 (Kerckhoffs's Principle)

- 现代密码学的重要原则
- 核心思想：
 - 算法可以公开
 - 安全性只依赖密钥

六条原则：

系统不需保密、
无需文字机密、
密钥易记易改、
适用于电讯、
方便携带、
使用简单

古典密码

- 简单代替密码
- 双重置换密码
- 一次一密
- 电码本密码

简单代替密码

- 简单代替密码：将明文按照字母表中当前字母后移n位加密而产生。

明文:	a	b	c	d	e	f	g	h	i	j	k	l	m	N	o	p	q	r	s	t	u	v	w	x	y	z
密文:	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C

这里n=3,
后移三位
形成

- 密文: **IRXUVFRUHDQGVHYHQBHDUVDJR**
- 明文: **fourscoreandsevenyearsago**

凯撒密码破译

- 凯撒密码：最早被凯撒在古罗马时期军事通信中，为了防止敌人截获信件，对信件进行简单加密。

明文:	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
密文:	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C

这里 $n=3$,
后移三位
形成

- 密文: **DGYDQFHG LQIRUPDWLRQ VHFXULWB**
- 明文: ?

密钥: n

- 将字母表向后平移 n 位, n 为某个特定值。 $n \in \{0, 1, 2, \dots, 25\}$.
- 简单替换密码里的关键内容就是 n 的值。
- 例如: key $n=7$.

明文:

a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G

密文:

密码破解1-穷举搜索

- 假设我们知道凯撒使用的密码
 - 将 n 平移 n , 其中 n 为某个特定值。
 - 但具体的 n 值未知。
- 给定密文: **CSYEVIXIVQMREXIH**
- 如何确定密钥?
- 只有 26 个可能的按键——试试所有按键!
- **穷举式密钥搜索**
- 解: 密钥 $n = 4$.  明文: YOUARETERMINATED

密码破解1-穷举搜索

- 一般来说，简单的替换键可以是字母的任意排列组合。
 - 不一定是凯撒密码（移位）
- 例如：

明文:	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
密文:	J	I	C	A	X	S	E	Y	V	D	K	W	B	Q	T	Z	R	H	F	M	P	N	U	L	G	O

- 总结来说，所有可能的情况有： $26! > 2^{88}$ 。
- $26! = 26 * 25 * 24 * \dots * 3 * 2 * 1$

密码破解1-穷举搜索

- 穷举搜索需要多久?
- 一个简单代替密码可以使用任何一个置换作为密钥:
- $26! > 2^{88}$ 种可能的密钥。
- 如果计算机每秒尝试 2^{40} 个密钥:
- 则需要: $T = \frac{2^{88}}{2^{40}} S = 2^{48} S = \frac{2^{48}}{60*60*24*365} \text{year} = ?$

网站输入账户密码时:
输错3次或者5次就锁住了。

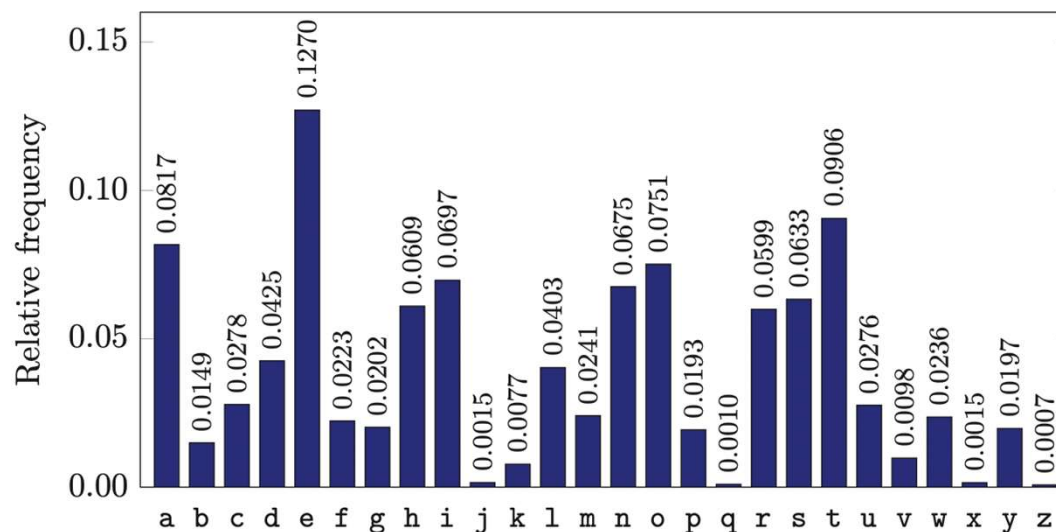
密码分析 2: 减少密钥搜索方法

- 我们知道密文在用简单替换密码
- 但是不知道密钥 $n=?$
- 那根据给定密文, 去分析 n 的值:

PBFPVYFBQXZTYFPBFEQJHDXXQVAPTPQJKTOYQWIPBVWLXTOXBTFXQWAXBVCXQWAXFQ
JWWLEQNTQZQGGQLFXQWAKVWLXQWAEBIPBFXFQVXGTVJWWLBTPQWAEBFPBFHCVLXB
QUFEVWLXGDPEQVPQGVPPBFTIXPFHXZHVFAGFOTHFEFBQUFTDHzBQPOTHXTYFTODX
QHFTDPTOGHFQPBQWAQJJDODXQHFOQPWTBDHHIXQVAPBFZQHCFWPFHPBFIPBQWKF
ABVYYDZBOTHBPBPQTQOTOGHFQAPBFEQJHDXXQVAVXEBQPEFZBVFOJIWFFACFCFHH
QWAUVWFLQHGFQVAFXQHFUFHILTTAVWAFFAWTEVOITDHFHFQAITIXPFHXAFQHEFZQW
GFLVWPTOFFA

密码分析2

- 无法尝试全部 $26! > 2^{88}$ 个简单替换键
- 我们还能更聪明些吗?
- 英文字母出现频率统计.....



密码分析2

- 给定密文:

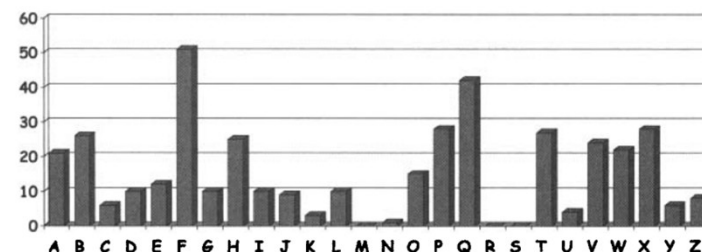
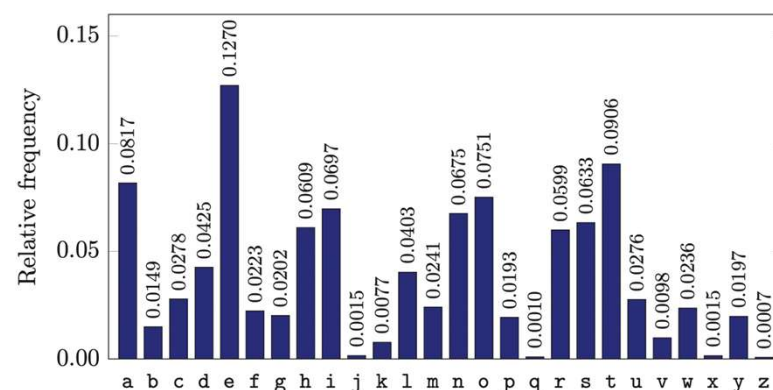
PBFPVYFBQXZTYFPBFEQJHDXXQVAPTPQJKTOYQWIPBVWLXTOXBTFFXQWAXBVCXQWAXFQ
 JWLEQNTQZQGGQLFXQWAKVWLXQWAEBIPBFXFQVXGTVJWVLBTPQWAEBFPBFHCVLXB
 QUFEVWLXGDPEQVPQGVPPBFTIXPFHXZHVFAGFOTHFEBQUFTDHBZBQPOTHTYFTODX
 QHFTDPTOGHFQPBQWAQJTDQXQHFOQPWTBDHHIXQVAPBFZQHCFWPFHPBFIPBQWKF
 ABVYYDZBOTHBPQPJTQOTOGHFQAPBFEQJHDXXQVAVXEBQPEFZBVFOJIWFFACFCFCH
 QWAUVWFLQHGFXVAFXQHUFHILTAVWAFFAWTEVOITDHFHFQAITIXPFHXAFQHEFZQW
 GFLWPTOFFA

- 统计密文的字母出现频率:

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
21	26	6	10	12	51	10	25	10	9	3	10	0	1	15	28	42	0	0	27	4	24	22	28	6	8

密码分析2:

- 根据密文统计, “F” 在密文中频率最高。
- 对照英文字母频率统计, “E” 频率最高。
- 结论: 最先尝试: $F \Rightarrow E$.
- 通过概论统计可以大大降低蛮力破解需要的时间。



密码安全性

- 如果已知最佳攻击是尝试所有密钥，则密码系统是**安全的**。
- 如果已知存在**任何捷径攻击**，则密码系统是**不安全的**。
- 但是，不安全的密码可能比安全的密码更难破解！

例子：维吉尼亚密码

- 简单替代密码是单表替换密码 (monoalphabetic).
- 维吉尼亚密码是多表替换密码 (polyalphabetic substitution)
- 凯撒密码可以基于一个关键词来进行移位操作。
- 例如：关键词CAT 表示：
 - C → 移位 2
 - A → 移位 0
 - T → 移位 19

例子：维吉尼亚密码

- 原文：HELLOWORLD
- 关键词：CAT 即移位2，移位0，移位19.
- 逐字加密规则：

明文：	H	E	L	L	O	W	O	R	L	D
密钥：	C	A	T	C	A	T	C	A	T	C

- 密文 = (明文+密钥) mod 26.

例子：维吉尼亚密码

- 最终密文：
JEENOPQREF
- 本质：多个凯撒密码
轮流使用
- 比凯撒密码更安全：
 - 每个字母移位不同
 - 频率分析不适用。

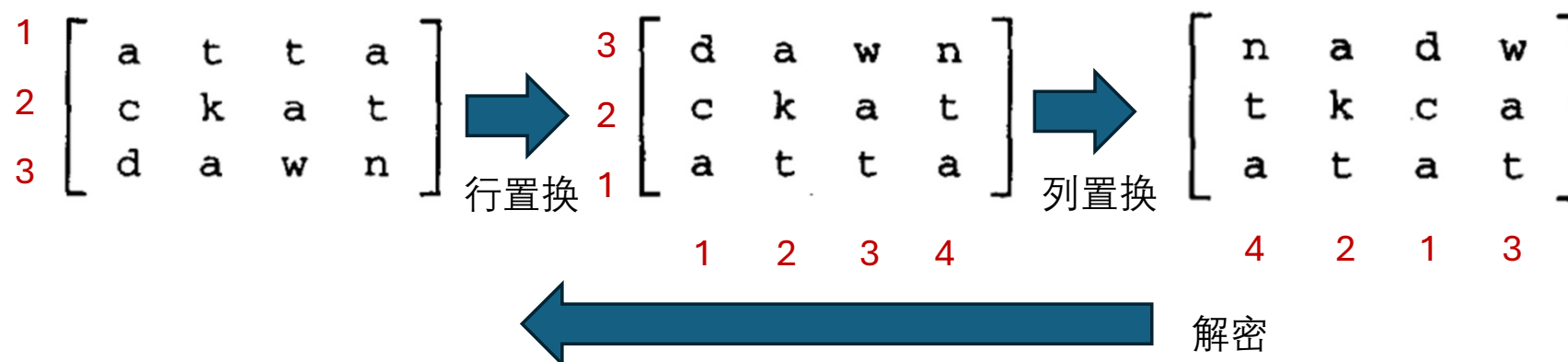
明文	数值	密钥	移位	计算	密文
H	7	C	2	$7+2=9$	J
E	4	A	0	$4+0=4$	E
L	11	T	19	$11+19=30 \bmod 26$	E
L	11	C	2	13	N
O	14	A	0	14	O
W	22	T	19	$41 \bmod 26$	P
O	14	C	2	16	Q
R	17	A	0	17	R
L	11	T	19	$30 \bmod 26$	E
D	3	C	2	5	F

双重置换密码

- 双重置换密码加密是将明文写成给定大小的矩阵形式，然后进行行置换和列置换。

• 明文: attackdawn  密文: NADWTKCAATAT

- 1. 行置换: $(1, 2, 3) \Rightarrow (3, 2, 1)$



双重置换密码

- 特点:
 - 字母不变
 - 字母频率不变
 - 比单次置换更安全
- 缺点:
 - 仍然可能被统计分析破解
 - 对短文本不安全，对长文本更安全

一次一密 (One-Time Pad)

- 一次一密：每个明文字符只使用一次随机密钥进行加密，并且密钥长度必须与明文相同。
- 规则：密文=明文 $\oplus$ 密钥 ($\oplus$ 为异或运算 XOR).

字母	e	h	i	k	l	r	s	t
二进制	000	001	010	011	100	101	110	111

- 明文转换为二进制串：001 000 010 100 001 010 111 100 000 101

一次一密 (One-Time Pad)

- 明文转换为二进制串: 001 000 010 100 001 010 111 100 000 101
- 选择对应长度的密钥: 111 101 110 101 111 100 000 101 110 000
- 然后明文和密钥加密运算 (按位异或操作),

	h	e	i	l	h	i	t	l	e	r
明文:	001	000	010	100	001	010	111	100	000	101
密钥:	111	101	110	101	111	100	000	101	110	000
密文:	110	101	100	001	110	110	111	001	110	101
	s	r	l	h	s	s	t	h	s	r

- 得到密文: srlhssthsr

一次一密 (One-Time Pad)

- 解密：使用同样的密钥解密（按位异或）

	s	r	l	h	s	s	t	h	s	r
密文：	110	101	100	001	110	110	111	001	110	101
密钥：	111	101	110	101	111	100	000	101	110	000
明文：	001	000	010	100	001	010	111	100	000	101
	h	e	i	l	h	i	t	l	e	r

VENONA项目

- VENONA 项目是在现实社会中的一个实例。
- 前苏联间谍使用一次一密的密钥进入美国，间谍使用密钥加密重要情报并传送至莫斯科。
- 美国截获密文并破译。
- 原因：产生密钥的过程中存在缺陷，使得密钥出现重复。

表 2.2 '1944 年 9 月 21 日的 VENONA 解密消息

[C% Ruth] learned that her husband [v] was called up by the army but he was not sent to the front. He is a mechanical engineer and is now working at the ENORMOUS [ENORMOZ] [vi] plant in SANTA FE, New Mexico.

[45 groups unrecoverable]

detain VOLOK [vii] who is working in a plant on ENORMOUS. He is a FELLOWCOUNTRYMAN [ZEMLYaK] [viii]. Yesterday he learned that they had dismissed him from his work. His active work in progressive organizations in the past was cause of his dismissal.

电码本密码

- 电码本密码：像字典一样的书，包含单词和其相应的码字。

表 2.3 德国电码本摘录

明文	密文
Februar	13605
fest	13732
finanzielle	13850
folgender	13918
Frieden	17142
Friedenschluss	17149
...	...

现代密码

- 古典密码 → 手工加密（字母替换、位置变化）
- 现代密码 → 数学算法 + 计算机计算
- 现代密码不仅用于加密通信，还用于：
 - 身份认证
 - 数据完整性
 - 数字签名
 - 网络安全

密码学发展历史

- 齐默尔曼电报
- 恩尼格玛密码机

密码学发展

- 第一个阶段是从古代到19世纪末——**古典密码 (classical cryptography)**
- 第二个阶段从20世纪初到1949年——**近代密码**
- 第三个阶段从C.E.Shannon (香农) 于1949年发表的划时代论文
“The Communication Theory of Secret Systems” 开始——**现代密码**
- 第四个阶段从1976年W. Diffie和M. Hellman创造性地发表了论文
“New Directions in Cryptography” 开始——**公钥密码**

齐默尔曼电报

- 史上最著名的密码本密文
- 美国卷入第一次世界大战的一个主要因素

WESTERN UNION TELEGRAM

NEWSPAPER CARLTON, PRESIDENT

Send the following telegram, subject to the terms on back hereof, which are hereby agreed to

via Galveston

JAN 18 1917

GERMAN LEGATION
MEXICO CITY

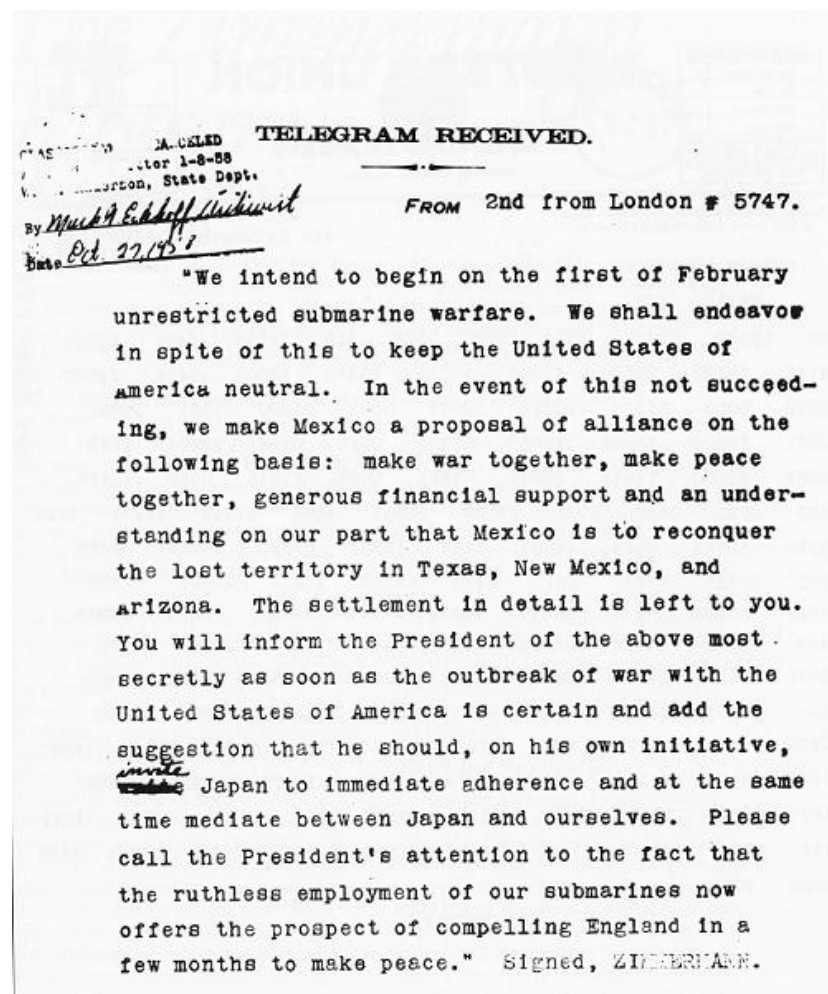
130	13042	13401	8501	115	3528	416	17214	8491	11310
18147	18222	21560	10247	11518	23677	13805	3494	14936	
98092	5905	11311	10392	10371	0302	21290	5161	39695	
23571	17504	11269	18276	18101	0317	0228	17694	4473	
23284	22200	19452	21589	87893	5569	13918	8958	12137	
1333	4725	4458	5905	17165	13851	4458	17149	14471	6706
13850	12224	6929	14991	7382	15857	67893	14218	36477	
5870	17553	67893	5870	5454	16102	15217	22801	17138	
21001	17388	7446	23638	18222	6719	14331	15021	23845	
3158	23552	22096	21804	4797	9497	22404	20855	4377	
23810	18140	22260	5905	13347	20420	39889	13732	20667	
0929	5275	18507	52262	1340	22049	13339	11265	22295	
10439	14814	4178	6992	8784	7632	7357	6926	52262	11267
21100	21272	9346	9559	22464	15874	18502	18500	15857	
2188	5376	7381	98092	16127	13486	9350	9220	76036	14219
5144	2831	17920	11347	17142	11264	7667	7762	15099	9110
10482	97556	3509	3670						

BEPNSTORFF.

Charge German Embassy.

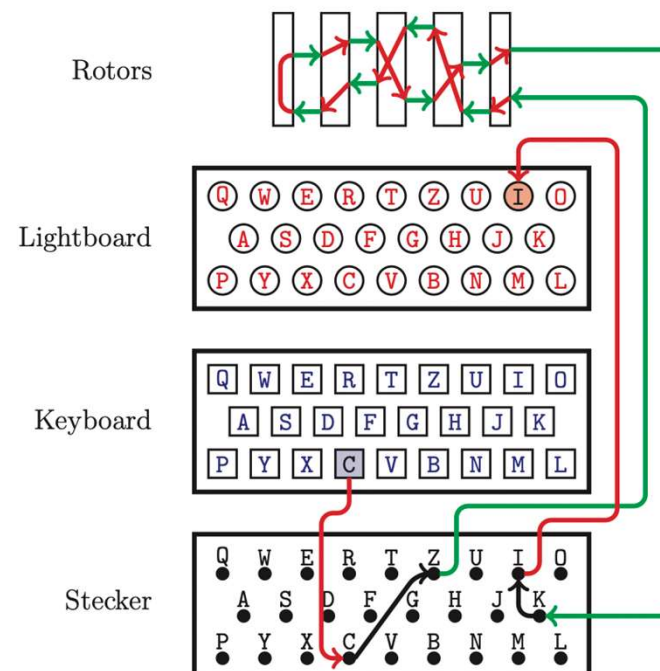
齐默尔曼电报解密

- ❑ 英国方面已找回部分密码本
- ❑ 然后能够填补缺失的部分



恩尼格玛密码机(Enigma)

- 二战中最著名的密码
 - 机电装置
 - 非常坚固耐用，野外使用。



二战后历史

- 克劳德·香农——信息论之父
- 计算机革命——大量数据需要保护
- 美国提出数据加密标准 (DES), 70年代
- 公钥密码学, 70年代
- 80年代的密码学会议
- 美国推出的高级加密标准 (AES), 90年代
- 密码学的潘多拉魔盒已经打开了.....

克劳德·香农

- 信息论的奠基领域
- 他1949年的论文: *Comm. Thy. of Secrecy Systems*
- 基本概念 (对称密码)
 - 混淆 (Confusion) ——明文与密文
 - 扩散 (Diffusion) ——通过密文
- 经证实, 一次一密是安全的。
- 一次一密仅使用混淆, 而双重移位仅使用扩散。

现代密码分类

- 对称密钥密码
- 非对称密钥密码 (公钥密码)
- 哈希函数

第二部分：对称密钥密码

对称密钥密码

- 流密码——一次一密的扩展
 - 只不过，关键部分相对较短。
 - 密钥被拉伸成一条长长的**密钥流**
 - Keystream 的使用方式与一次一密类似。
- 分组密码——通用密码本
 - 分组密码 **密钥** 确定密码本
 - 每个密钥对应不同的密码本。
 - 同时运用了“混淆”和“扩散”策略

流密码

- 流密码：按位或者按字节逐个加密数据流。
- 是前面古典密码中一次一密的扩展。
- 我们将讨论两种流密码：
- A5 / 1
 - 基于移位寄存器
 - 用于GSM移动电话系统无线通信保护
- RC4
 - 基于不断变化的查找表
 - 很多地方都用过（例如WEP）

一次一密

密钥完全随机

密钥长度=明文长度

绝对安全

流密码

密钥流伪随机

密钥较短

计算安全

A5/1 流密码

- 一次一密：密钥完全随机
- A5 / 1：使用伪随机的密钥
- 伪随机密钥如何生成：

使用基于三个寄存器的伪随机数生成器。

- 通过三个寄存器的 X18, Y21, Z22 三个位异或产生一位密钥流。

一次一密

密钥完全随机

密钥长度=明文长度

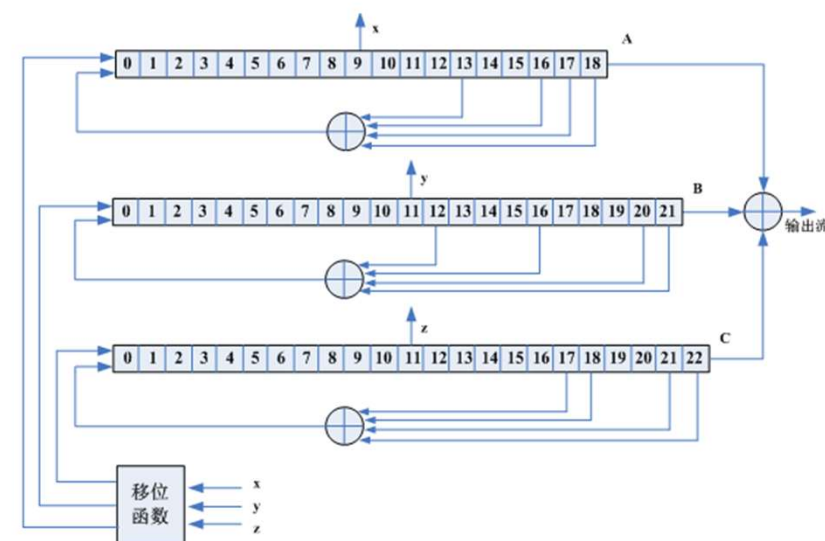
绝对安全

流密码

密钥流伪随机

密钥较短

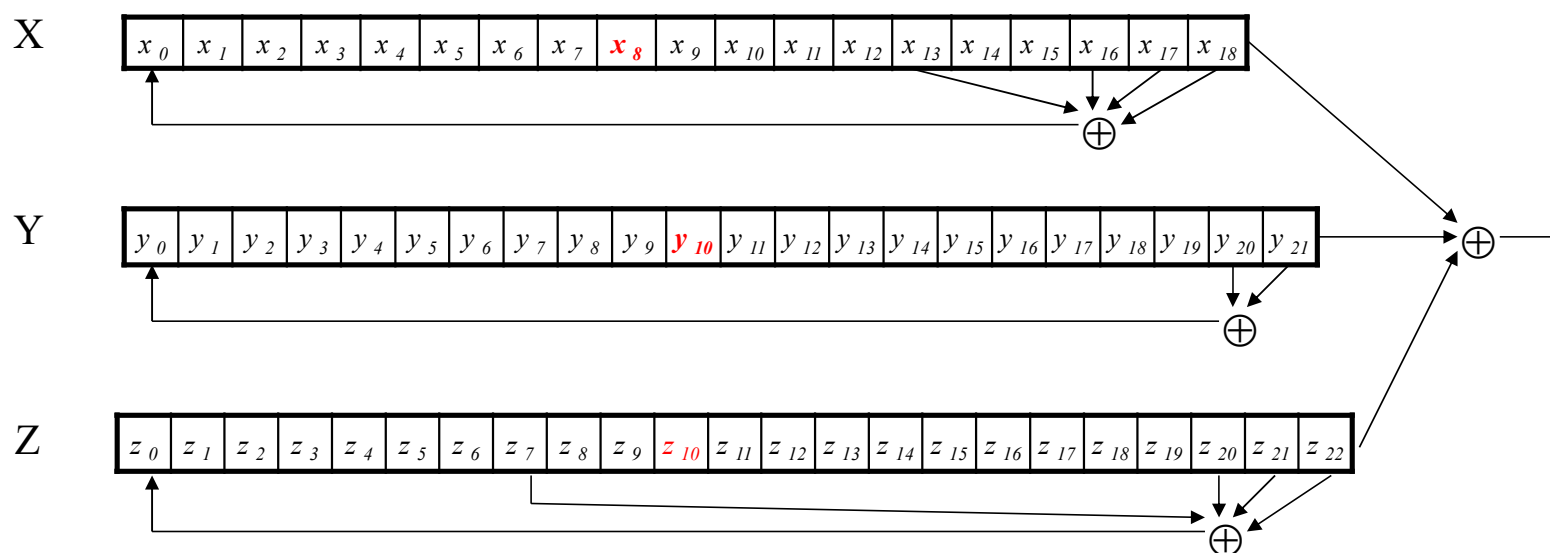
计算安全



A5/1 流密码

- 如何生成密钥流?
 - 初始化3个移位寄存器
 - X: 19位 ($x_0, x_1, x_2, \dots, x_{18}$)
 - Y: 22位 ($y_0, y_1, y_2, \dots, y_{21}$)
 - Z: 23位 ($z_0, z_1, z_2, \dots, z_{22}$)
 - 钟控机制
 - 选择 X_8, Y_{10}, Z_{10} 三位做选择 控制每个寄存器是否移位。
 - 生成序列
 - 每次做钟控后, 对应寄存器操作, 之后对三个寄存器最高位, 异或操作产生新的一位密钥流。

A5/1: 流密码



- 这里每个变量都是一个bit。
- 密钥用作寄存器的**初始填充值**。
- 每个寄存器根据 $\text{maj}(x_8, y_{10}, z_{10})$ 进行步进（或不步进）。
- 密钥流位是寄存器最右侧几位的异或结果。

A5/1: 流密码

- 钟控操作:

- 少数服从多数: 在每次迭代中: $m = \text{majority}(X_8, Y_{10}, Z_{10})$

- 例如: $\text{majority}(0,1,0) = 0$ 和 $\text{majority}(1,1,0) = 1$

- 如果 $X_8 = m$, 则X 寄存器进行移位操作:

- $t = x_{13} \oplus x_{16} \oplus x_{17} \oplus x_{18}$
 - $x_i = x_{i-1}$ for $i = 18, 17, \dots, 1$ and $x_0 = t$

- 如果 $Y_{10} = m$, 则Y 寄存器进行移位操作

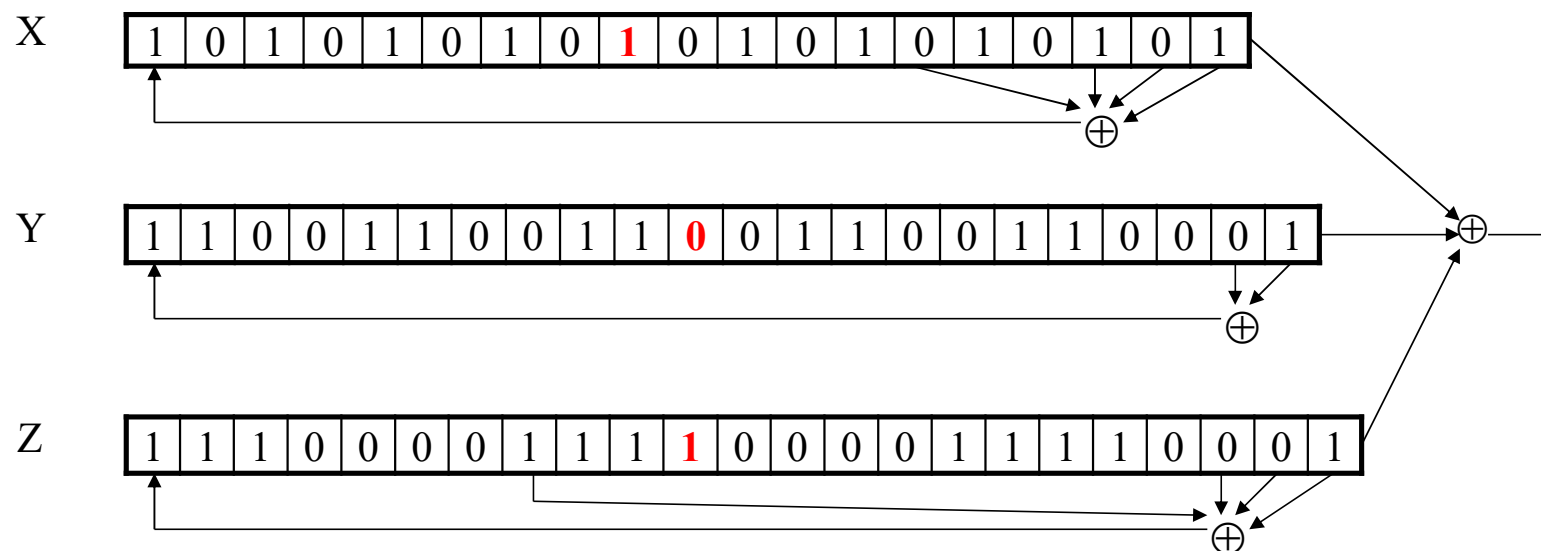
- $t = y_{20} \oplus y_{21}$
 - $y_i = y_{i-1}$ for $i = 21, 20, \dots, 1$ and $y_0 = t$

- 如果 $Z_{10} = m$, 则Z 寄存器进行移位操作

- $t = z_7 \oplus z_{20} \oplus z_{21} \oplus z_{22}$
 - $z_i = z_{i-1}$, 其中 $i = 22, 21, \dots, 1$, 且 $z_0 = t$

- 上述操作完成后, 由三个寄存器的最高位进行异或操作, $x_{18} \oplus y_{21} \oplus z_{22}$ 生成密钥流的一位。

A5/1: 流密码



- 在这个例子中, $m = \text{maj}(x_8, y_{10}, z_{10}) = \text{maj}(1, 0, 1) = 1$
- 注册X步进, Y不步进, Z步进
- 密钥流位是寄存器右位的异或结果。
- 这里, 密钥流位将为 $0 \oplus 1 \oplus 0 = 1$

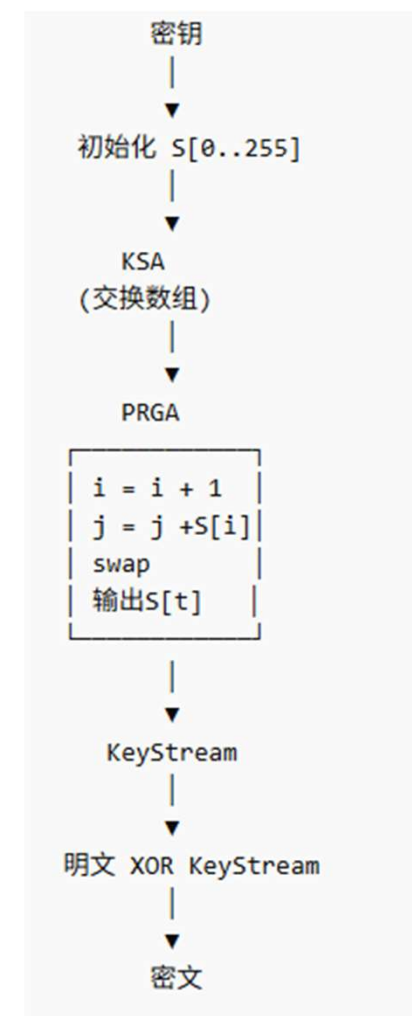
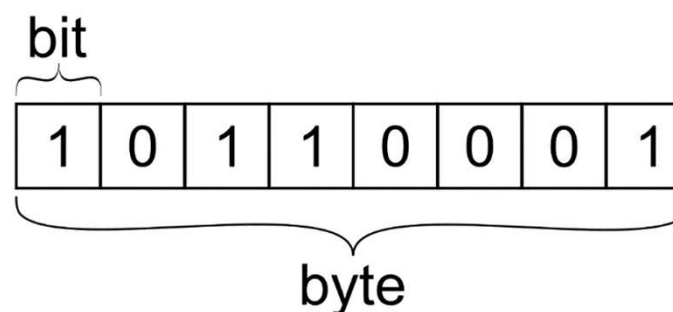
A5/1: 流密码

- 移位寄存器在硬件上具有加密效率
- 如果用软件实现，通常速度很慢。
- 过去非常非常受欢迎
- 如今，由于处理器速度更快，越来越多的工作通过软件完成。
- 移位寄存器加密仍然被使用了一些
 - 尤其是在资源受限的设备中

RC4 流密码

- “Rivest Code 4” (RC4)
- RC4 算法专门为软件实现优化。
 - A5/1: 每步产生一个bit密钥。
 - RC4: 每步产生一个byte (字节)。
- 算法是基于字节的

密文 = 明文 XOR 密钥流



RC4 流密码

- $S[]$ 是 $0,1,\dots,255$ 的256字节的列表。
- $key[]$ 包含 N 个字节的密钥

初始化

```
for  $i = 0$  to 255  
     $S[i] = i$   
     $K[i] = key[i \bmod N]$   
next  $i$   
 $j = 0$   
for  $i = 0$  to 255  
     $j = (j + S[i] + K[i]) \bmod 256$   
    swap( $S[i], S[j]$ )  
next  $i$   
 $i = j = 0$ 
```

表 3.2 RC4 密钥流字节

```
 $i = (i + 1) \bmod 256$   
 $j = (j + S[i]) \bmod 256$   
swap( $S[i], S[j]$ )  
 $t = (S[i] + S[j]) \bmod 256$   
keystreamByte =  $S[t]$ 
```

分组密码

- Feistel 密码结构
- DES
- 三重DES
- AES
- 其他三组分组密码

分组密码

- 分组密码将明文分为固定长度的分组，并产生固定长度的密文分组。
- 明文通过迭代**轮函数(round)**获得密文。
- 轮询函数的输入是上一轮的输出和密钥K。

费斯特尔密码 (Feistel)

- 以先驱Horst Feistel 名字命名。
- 是密码设计的一个主要原则，一种结构，而不是特殊的密码。
- 明文P 被分为左右两部分，

$$P = (L_0, R_0)$$

- 然后，左右部分操作 $L_i = R_{i-1}$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

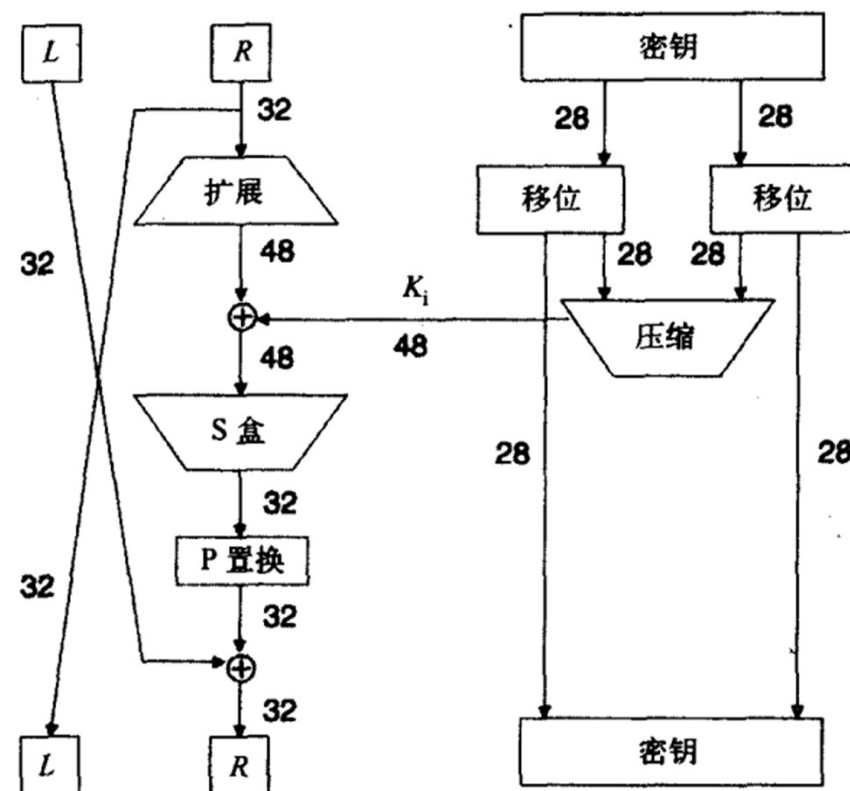
- 这里 K_i 是第 i 轮的子密钥 (subkey) 。子密钥由密钥 K 通过密钥扩展算法产生。最后输出密文: $C = (L_n, R_n)$

DES

- DES: Data Encryption Standard, 数据加密标准。
- 基于 IBM 的 Lucifer 密码,
- DES是美国国家标准局（NIST）征集的IBM的Lucifer算法经过国家安全局（NSA）设计优化的。
- 主要更改：
 - 密钥长度从 128 位减少到 56 位
 - 对S盒的置换

DES

- DES是16轮的Feistel结构密码;
- DES的分组长度是64位;
- DES使用56位的密钥
- DES的每一轮使用48位的子密钥,
- 每个子密钥都由56位密钥子集构成。
- 轮函数很简单 (对于分组密码而言)
- 安全很大程度上依赖于“S盒”。
 - 8个S盒中的每一个都将6位映射到4位



DES

- DES 函数F 是扩展置换:

$$F(R_{i-1}, K_i) = \text{P-box}(\text{S-boxes}(\text{Expand}(R_{i-1}) \oplus K_i))$$

(P-box: P 置换; S-boxes: S 盒; Expand: 扩展)

DES 扩展“排列”

- 输入 32 位

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

- 输出 48 位

31 0 1 2 3 4 3 4 5 6 7 8
7 8 9 10 11 12 11 12 13 14 15 16
15 16 17 18 19 20 19 20 21 22 23 24
23 24 25 26 27 28 27 28 29 30 31 0

DES S 盒

- 8 个“替换框”或 S 框。
- 每个 S 盒将 6 位映射到 4 位，将 48 位通过 S 盒转换为 32 位。
- 这是 S 盒 1 号。

输入位 (0, 5)

↓ 输入位 (1, 2, 3, 4)

	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111

00	1110	0100	1101	0001	0010	1111	1011	1000	0011	1010	0110	1100	0101	1001	0000	0111
01	0000	1111	0111	0100	1110	0010	1101	0001	1010	0110	1100	1011	1001	0101	0011	1000
10	0100	0001	1110	1000	1101	0110	0010	1011	1111	1100	1001	0111	0011	1010	0101	0000
11	1111	1100	1000	0010	0100	1001	0001	0111	0101	1011	0011	1110	1010	0000	0110	1101

DES P-box

- 输入 32 位

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

- 输出 32 位

15 6 19 20 28 11 27 16 0 14 22 25 4 17 30 9
1 7 23 13 31 26 2 8 18 12 29 5 21 10 3 24

DES 子密钥

- 56 位 DES 密钥, 编号为 0,1,2,...,55

- 左半键位, LK

49 42 35 28 21 14 7

0 50 43 36 29 22 15

8 1 51 44 37 30 23

16 9 2 52 45 38 31

- 右半键位, RK

55 48 41 34 27 20 13

6 54 47 40 33 26 19

12 5 53 46 39 32 25

18 11 4 24 17 10 3

DES 子密钥

- 对于第 $i = 1, 2, \dots, 16$ 轮
 - 令 $LK = (LK \text{ 左移循环移位 } r_i)$
 - 令 $RK = (RK \text{ 左移循环移位 } r_i)$
 - 子键的左半部分 K_i 是 LK 位

13 16 10 23 0 4 2 27 14 5 20 9
22 18 11 3 25 7 15 6 26 19 12 1

- 子键的右半部分 K_i 是 RK 位

12 23 2 8 18 26 1 11 22 16 4 19
15 20 10 27 5 24 17 13 21 7 0 3

DES 子密钥

- 第 1、2、9 和 16 轮的偏移量 r_i 为 1，其他所有轮次的偏移量 r_i 为 2。
- LK 的第 8、17、21、24 位在每一轮中都被省略。
- RK 的每一轮都省略了第 6、9、14、25 位比特。
- **压缩置换** 产生 48 位子密钥 K_i 由 56 位 LK 和 RK 组成。
- **密钥调度算法** 生成子密钥

DES的安全性

- 安全很大程度上依赖于S盒。
 - DES 中的其他一切都是线性的。
- 超过45年的分析表明，没有后门
- 攻击？本质上是穷举式密钥搜索
- **不可避免的结论**
 - DES的设计者们很清楚自己在做什么。
 - DES的设计者们远远领先于他们的时代（至少在某些密码分析技术方面是如此）。

三重DES

- 56 位 DES 密钥太小了。
 - 穷举密钥搜索是可行的。
- 但 DES 无处不在，那该怎么办？
- **三重DES**或**3DES**（112位密钥）
 - $C = E(D(E(P, K_1), K_2), K_1)$
 - $P = D(E(D(C, K_1), K_2), K_1)$
- 为什么要使用两个密钥进行加密-解密-加密？
 - 向后兼容： $E(D(E(P, K), K), K) = E(P, K)$
 - 112 是很大的比特数。

三重DES

- 为什么不改用 $C = E(E(P, K), K)$ 呢?
 - 这是一个陷阱题，因为它仍然只是一个 56 位密钥。
- 为什么不改用 $C = E(E(P, K_1), K_2)$ 呢?
- 一种（半实用的）**已知明文**攻击
 - 预先计算 $E(P, K_1)$ 表，针对每个可能的键 K_1 （结果表有 2^{56} 个条目）
 - 然后，对于每个可能的 K_2 ，计算 $D(C, K_2)$ ，直到在表中找到匹配项。
 - 当找到匹配项时，有 $E(P, K_1) = D(C, K_2)$
 - 结果给出的键为： $C = E(E(P, K_1), K_2)$

AES

- DES의替代品
- AES竞赛 (90年代末)
 - 美国国家安全局公开参与其中
 - 透明的遴选过程
 - 许多强大的算法被提出
 - Rijndael算法 (发音类似“Rain Doll”或“Rhine Doll”)
- 迭代分组密码 (例如 DES)
- 不是费斯特尔密码 (与DES不同)

AES

- 支持三种分组长度：128位，192位，和256位。
- 支持三种密钥长度：128位，192位，和256位。
- 轮数从10~14不等，由密钥长度决定。
- 每轮包括四个函数，划分三个层次：
 - ByteSub (非线性层)
 - ShiftRow (线性混淆层)
 - MixColumn (线性混淆层)
 - AddRoundKey (密钥加层)

AES 字节串

- 将 192 位块视为 4x6字节数组

$$\begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} & a_{04} & a_{05} \\ a_{10} & a_{11} & a_{12} & a_{13} & a_{14} & a_{15} \\ a_{20} & a_{21} & a_{22} & a_{23} & a_{24} & a_{25} \\ a_{30} & a_{31} & a_{32} & a_{33} & a_{34} & a_{35} \end{bmatrix} \xrightarrow{\text{ByteSub}} \begin{bmatrix} b_{00} & b_{01} & b_{02} & b_{03} & b_{04} & b_{05} \\ b_{10} & b_{11} & b_{12} & b_{13} & b_{14} & b_{15} \\ b_{20} & b_{21} & b_{22} & b_{23} & b_{24} & b_{25} \\ b_{30} & b_{31} & b_{32} & b_{33} & b_{34} & b_{35} \end{bmatrix}$$

- ByteSub是 AES 的“S 盒”。
- 可以看作是两个数学运算的非线性（但可逆）组合。

AES의ByteSub

- AES의ByteSub。
- Bytesub操作可以视为一个简单的查表。

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
1	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
2	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
3	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
4	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
5	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
6	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
7	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
8	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
9	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
a	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
b	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
c	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
d	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
e	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
f	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

AES ShiftRow

- ShiftRow 操作是对字节矩阵进行按字节循环移位

$$\begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} & a_{04} & a_{05} \\ a_{10} & a_{11} & a_{12} & a_{13} & a_{14} & a_{15} \\ a_{20} & a_{21} & a_{22} & a_{23} & a_{24} & a_{25} \\ a_{30} & a_{31} & a_{32} & a_{33} & a_{34} & a_{35} \end{bmatrix} \xrightarrow{\text{ShiftRow}} \begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} & a_{04} & a_{05} \\ a_{11} & a_{12} & a_{13} & a_{14} & a_{15} & a_{10} \\ a_{22} & a_{23} & a_{24} & a_{25} & a_{20} & a_{21} \\ a_{33} & a_{34} & a_{35} & a_{30} & a_{31} & a_{32} \end{bmatrix}$$

AES MixColumn

- 对每一列应用可逆线性运算

$$\begin{bmatrix} a_{0i} \\ a_{1i} \\ a_{2i} \\ a_{3i} \end{bmatrix} \rightarrow \text{MixColumn} \rightarrow \begin{bmatrix} b_{0i} \\ b_{1i} \\ b_{2i} \\ b_{3i} \end{bmatrix}, i = 0, 1, 2, \dots, 5$$

- 实现方式为（大型）查找表

AES 的AddRoundKey

- 将子密钥与当前矩阵块进行异或操作。

$$\begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} & a_{04} & a_{05} \\ a_{10} & a_{11} & a_{12} & a_{13} & a_{14} & a_{15} \\ a_{20} & a_{21} & a_{22} & a_{23} & a_{24} & a_{25} \\ a_{30} & a_{31} & a_{32} & a_{33} & a_{34} & a_{35} \end{bmatrix} \oplus \begin{bmatrix} k_{00} & k_{01} & k_{02} & k_{03} & k_{04} & k_{05} \\ k_{10} & k_{11} & k_{12} & k_{13} & k_{14} & k_{15} \\ k_{20} & k_{21} & k_{22} & k_{23} & k_{24} & k_{25} \\ k_{30} & k_{31} & k_{32} & k_{33} & k_{34} & k_{35} \end{bmatrix} = \begin{bmatrix} b_{00} & b_{01} & b_{02} & b_{03} & b_{04} & b_{05} \\ b_{10} & b_{11} & b_{12} & b_{13} & b_{14} & b_{15} \\ b_{20} & b_{21} & b_{22} & b_{23} & b_{24} & b_{25} \\ b_{30} & b_{31} & b_{32} & b_{33} & b_{34} & b_{35} \end{bmatrix}$$

- 密钥调度算法确定的轮密钥（子密钥）

AES解密

- 要解密，该过程必须是可逆的。
- AddRoundKey的逆运算很简单，因为“ $\oplus$ ”本身就是它的逆运算。
- MixColumn是可逆的（逆操作也以查找表的形式实现）。
- ShiftRow的逆操作很简单（向相反方向循环移位）。
- ByteSub是可逆的（逆运算也是用查找表实现的）。

TEA

- 微型加密T算法 (TEA)
- 64 位块, 128 位密钥
- 假设采用 32 位运算
- 回合数可变 (32 回合被认为是安全的)
- 使用“弱”轮函数, 因此需要大量的轮数。

TEA加密

表 3.5 TEA 加密过程

```
(K[0], K[1], K[2], K[3]) = 128 bit key
(L, R) = plaintext (64-bit block)
delta = 0x9e3779b9
sum = 0
for i = 1 to 32
    sum = sum + delta
    L = L + (((R << 4) + K[0]) ⊕ (R + sum) ⊕ ((R >> 5) + K[1]))
    R = R + (((L << 4) + K[2]) ⊕ (L + sum) ⊕ ((L >> 5) + K[3]))
next i
ciphertext = (L, R)
```

TEA解密

表 3.6 TEA 解密过程

```
(K[0], K[1], K[2], K[3]) = 128 bit key
(L, R) = ciphertext (64-bit block)
delta = 0x9e3779b9
sum = delta << 5
for i = 1 to 32
    R = R - (((L << 4) + K[2]) ⊕ (L + sum) ⊕ ((L >> 5) + K[3]))
    L = L - (((R << 4) + K[0]) ⊕ (R + sum) ⊕ ((R >> 5) + K[1]))
    sum = sum - delta
next i
plaintext = (L, R)
```

TEA

- “几乎”是费斯特尔密码
 - 使用+和-操作 ($\text{mod } 2^{32}$) 而不是 $\oplus$ (XOR)
- 简单、易于实施、速度快、内存占用低等。
- 可能是“关联密钥”攻击
- 扩展型TEA (XTEA) 消除了相关密钥攻击 (稍微复杂一些)。
- 简化版TEA (STEAL) ——不安全版本, 用作密码分析示例

分组密码工作模式

- ECB (电码本模式)
- CBC (密文链接模式)
- CTR (计数器模式)

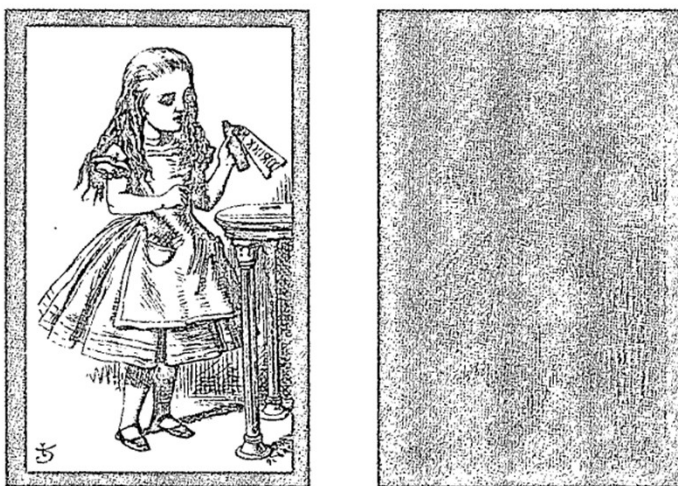


图 3.3 Alice 和 ECB 模式

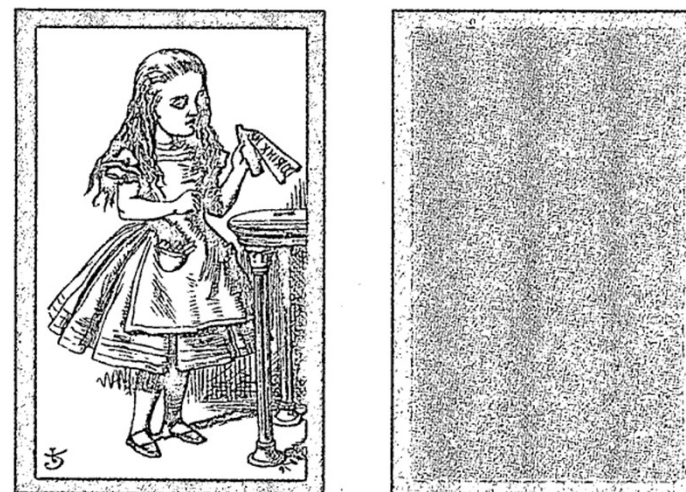


图 3.4 Alice 和 CBC 模式

计数器模式 (CTR)

- CTR 因其随机访问功能而广受欢迎
- 使用分组密码，就像使用流密码一样。

加密

$$C_0 = P_0 \oplus E(IV, K)$$

$$C_1 = P_1 \rightarrow E(IV+1, K)$$

$$C_2 = P_2 \rightarrow E(IV+2, K)$$

解密

$$P_0 = C_0 \oplus E(IV, K)$$

$$P_1 = C_1 \rightarrow E(IV+1, K)$$

$$P_2 = C_2 \rightarrow E(IV+2, K)$$

- 注：CBC 也适用于随机访问
 - 但这种情况存在一个重大局限性。

完整性

- 秘密性是阻止非授权的读取。
- 完整性是阻止非授权的写。
- 例如：银行间资金转账
 - 保密固然重要，但诚信才是**关键**。
- 加密技术提供**保密性**（防止未经授权的披露）。
- 仅靠加密**无法**提供完整性。
 - 一次性便签纸、剪贴簿等等。

MAC 消息认证码

- 消息认证码MAC 使用分组密码确保数据完整性。
- 与CBC模式加密数据类似。
- 只保留最后一块密文分组作为MAC。
- 假设有N块分组：

$$C_0 = E(P_0 \oplus IV, K), \quad C_1 = E(P_1 \oplus C_0, K), \dots$$
$$C_{N-1} = E(P_{N-1} \oplus C_{N-2}, K) = \text{MAC}$$

- 接收者收到数据，重复计算过程，并比较新计算出的MAC与接收到的MAC是否一致。

对称密钥密码

发送方和
接收方都
必须知道
K

对称加密

- 流密码: A5/1和RC4
- 分组密码: feistel 密码结构, DES, AES 和其他密码
- 完整性 (MAC)

Next week

- 非对称加密 (公钥密码)
- Hash函数
- 高级密码分析



Thank you!